

JavaScript Array Method - includes()

The `includes()` method is one of the most useful array methods in JavaScript. It is used to check whether an array contains a specific element.

Unlike `indexOf()`, `includes()` returns a **Boolean value** (`true` or `false`), making it simple and readable.

Table of Contents

- What is includes()?
- Why Use includes()?
- Syntax
- Parameters
- Return Value
- How includes() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is includes()?

Definition

The `includes()` method checks whether a specified element exists inside an array.

If the element exists, it returns

```
true
```

Otherwise, it returns

```
false
```

It **does not modify** the original array.

Why Use includes()?

We use `includes()` when we need to:

- Check if an item exists
- Validate user input
- Check permissions
- Verify selected products
- Prevent duplicate values
- Search simple values in an array

Syntax

```
array.includes(value);  
  
array.includes(value, startIndex);
```

Parameters

Parameter	Description
value	The value to search for
startIndex	Optional. Index to start searching from

Return Value

Returns

```
true
```

if the value exists.

Returns

```
false
```

if the value does not exist.

How includes() Works

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
includes("Mango")
```

↓

Result

```
true
```

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
includes("Banana")
```

↓

Result

```
false
```

Example 1 (Basic)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.includes("Mango"));
```

Output

```
true
```

Explanation

"Mango" exists inside the array.

Example 2 (Value Not Found)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.includes("Banana"));
```

Output

```
false
```

Example 3 (Numbers)

```
const numbers = [  
  10,  
  20,  
  30,
```

```
    40  
  
];  
  
console.log(numbers.includes(30));
```

Output

```
true
```

Example 4 (Boolean)

```
const values = [  
  true,  
  false  
];  
  
console.log(values.includes(true));
```

Output

```
true
```

Example 5 (Using startIndex)

```
const numbers = [
```

```
    10,  
    20,  
    30,  
    40  
];  
  
console.log(numbers.includes(20,2));
```

Output

```
false
```

Explanation

Searching starts from index **2**.

Index 2 contains

```
30
```

So

```
20
```

is not found.

Example 6 (Empty Array)

```
const data = [];  
  
console.log(data.includes(100));
```

Output

```
false
```

Example 7 (Case Sensitive)

```
const fruits = [  
  "Apple",  
  "Mango"  
];  
  
console.log(fruits.includes("apple"));
```

Output

```
false
```

Explanation

JavaScript is case-sensitive.


```
Apple
```

is different from

```
apple
```

Example 8 (Duplicate Values)

```
const numbers = [  
  10,  
  20,  
  20,  
  30  
];  
  
console.log(numbers.includes(20));
```

Output

```
true
```

Real Project Example 1

Shopping Cart

```
const cart = [  
  "Laptop",  
  "Mouse"  
];  
  
console.log(cart.includes("Laptop"));
```

Output

```
true
```

Real Project Example 2

User Role

```
const roles = [  
  "Admin",  
  "Editor",  
  "User"  
];  
  
console.log(roles.includes("Admin"));
```

Output

```
true
```

Real Project Example 3

Prevent Duplicate Student

```
const students = [  
  "Rahim",  
  "Karim"  
];  
  
const newStudent = "Rahim";  
  
if(!students.includes(newStudent)){  
  students.push(newStudent);  
}  
  
console.log(students);
```

Output

```
["Rahim","Karim"]
```

Real Project Example 4

Allowed Countries

```
const countries = [  
  "Bangladesh",  
  "India",  
  "Japan"  
];  
  
console.log(countries.includes("Bangladesh"));
```

Output

```
true
```

Real Project Example 5

Favorite Programming Languages

```
const languages = [  
  "JavaScript",  
  "Python",  
  "Java"  
];  
  
console.log(languages.includes("Python"));
```

Output

```
true
```

Best Practices

Use `includes()` when you only need to know whether a value exists.

Use meaningful variable names.

```
products
```

```
students
```

```
roles
```

```
languages
```

Use `includes()` instead of `indexOf()` for readability.

```
if(products.includes("Laptop")){  
  
}
```

Common Mistakes

Mistake 1

Expecting `includes()` to return an index.

Wrong

```
const result = numbers.includes(20);

console.log(result);
```

Output

```
true
```

It returns a Boolean, **not an index**.

Mistake 2

Ignoring case sensitivity.

```
const fruits = ["Apple"];

console.log(fruits.includes("apple"));
```

Output

```
false
```

Mistake 3

Using `includes()` to find an object.

```
const users = [

  {
```

```
        name: "John"
    }
];

console.log(users.includes({
    name: "John"
}));
```

Output

```
false
```

Objects are compared by **reference**, not by value.

Mistake 4

Confusing `includes()` with `indexOf()` .

```
includes()
```

returns

```
true/false
```

while

```
indexOf()
```

returns

```
index / -1
```

Interview Questions

What does `includes()` do?

Checks whether a value exists in an array.

What does `includes()` return?

```
true
```

or

```
false
```

Does `includes()` modify the original array?

No.

Is `includes()` case-sensitive?

Yes.

Can `includes()` search from a specific index?

Yes.

```
array.includes(value, startIndex);
```

Which method returns an index?

```
indexOf()
```

Which method returns true or false?

```
includes()
```

Summary Table

Feature	Description
Method	<code>includes()</code>
Purpose	Check whether a value exists
Parameters	value, startIndex (optional)
Returns	<code>true</code> or <code>false</code>
Changes Original Array	No
Common Uses	Validation, Search, Permissions, Duplicate Check

Quick Comparison

Method	Returns	Modifies Array
<code>includes()</code>	<code>true</code> / <code>false</code>	No
<code>indexOf()</code>	Index / <code>-1</code>	No
<code>lastIndexOf()</code>	Last Index / <code>-1</code>	No

Final Notes

The `includes()` method is one of the easiest and most readable ways to check if an array contains a value. It is widely used in:

- Form validation
- Shopping cart applications
- Authentication & authorization
- Duplicate prevention
- Search functionality
- Permission checking

Mastering `includes()` will make your JavaScript code cleaner, more readable, and easier to maintain.